

Pourquoi ça plafonne ?

acte 1 : le symptôme

à lire : environ 52 s · cumul ≈ 0 min 52 s

1 Symptôme

2 Enquête

3 Remède

4 Charge

5 Prix

Fil rouge : Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent. D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

UNE PAGE DE 20 COMMANDES = 21 ALLERS-RETOURS**≈ 247**

req/s : plafond dès 5 clients

98 ms

latence p95 à 20 clients

15

requêtes en attente d'une connexion

À DIRE

Bonjour. Je vais vous raconter une enquête, avec une seule question : **pourquoi une page de 20 commandes devient-elle lente quand les clients arrivent ?**

Imaginez une boutique en ligne : pour afficher une page de 20 commandes, le serveur fait **21 allers-retours** vers la base, un pour la liste, puis un par commande. Ça marche, mais sous charge ça coince : avec 20 clients, le débit plafonne à environ **247 requêtes par seconde** dès 5 clients, le p95 monte à **98 millisecondes**, et 15 requêtes attendent une connexion. Aucune erreur : le service est simplement saturé. Le symptôme est clair. Reste à trouver d'où vient ce coût, et c'est l'objet de l'enquête.

Où est le coupable ?

acte 2 : l'enquête

à lire : environ 47 s · cumul ≈ 1 min 39 s

- 1 Symptôme
- 2 Enquête
- 3 Remède
- 4 Charge
- 5 Prix

Fil rouge : Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent. D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

LES SUSPECTS

Suspect	Verdict	Preuve
La requête SQL	innocente	0,018 ms vue par PostgreSQL ; absente du journal des requêtes lentes
Le pool de connexions	conséquence	file jusqu'à 15 ; pool 20 : ≈ 287 req/s seulement
Les allers-retours	coupable	21 par page ; 0,68 ms chacun côté API

OÙ EST LE COÛT ? (TEMPS PAR REQUÊTE DE LIGNES)



À DIRE

Premier suspect : la requête elle-même. Elle est **innocente** : PostgreSQL la voit à 0,018 milliseconde, elle ne ressort pas en tête de `pg_stat_statements`, et elle n'apparaît pas dans le journal des requêtes lentes.

Deuxième suspect : le pool de connexions. Sa file d'attente est une **conséquence**, pas la cause : même avec un pool de 20, on n'atteint que 287 requêtes par seconde. Le vrai coupable, ce sont **les allers-retours** : côté API, chaque requête coûte 0,68 milliseconde, soit **38 fois plus** que ce que voit PostgreSQL. Multipliez ce prix par 21 pour chaque page, et vous obtenez le plafond.

Peut-on le supprimer sans rien casser ?

acte 3 : le remède

à lire : environ 53 s · cumul ≈ 2 min 32 s

1 Symptôme

2 Enquête

3 Remède

4 Charge

5 Prix

Fil rouge : Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent. D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

ANCIENNE VERSION

```
rows = SELECT ... commandes
        LIMIT 20;
POUR CHAQUE commande :
  SELECT ... FROM lignes
    WHERE commande_id = $1;
-- 1 + 20 = 21 requêtes
```

NOUVELLE VERSION

```
rows = SELECT ... commandes
        LIMIT 20;
SELECT ... FROM lignes
  WHERE commande_id =
    ANY($1::bigint[]);
-- $1 = les 20 ids
-- 2 requêtes, toujours
```

MÊME RÉSULTAT ✓

JSON complet identique à 5, 20, 50 et 100 commandes · droits conservés (401 / 400 / 400)
21 requêtes → 2, quelle que soit la page ; un seul changement.

À DIRE

Le remède tient en une idée : au lieu d'une requête par commande, **une seule requête pour toutes les lignes de la page**, avec ANY et la liste des identifiants. On passe de 21 à **2 requêtes**, quelle que soit la taille de la page. C'est un seul changement : je ne touche ni à l'index, ni au pool, ni à la base. Mais un remède qui casse le résultat ne vaut rien. Je vérifie donc : le JSON complet, commandes et lignes, est **identique** avec 5, 20, 50 et 100 commandes, et les droits sont conservés : 401 sans jeton, 400 si on injecte un client dans l'URL. Même résultat, moins de travail.

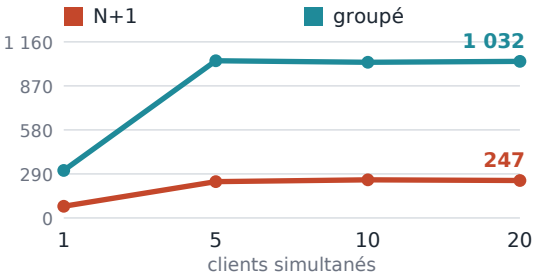
Le gain tient-il sous charge ?

acte 4 : la preuve à la charge
à lire : environ 52 s · cumul ≈ 3 min 24 s

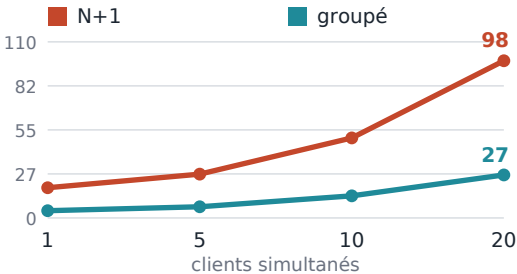
- 1 Symptôme
- 2 Enquête
- 3 Remède
- 4 Charge
- 5 Prix

Fil rouge : Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent. D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

Requêtes terminées par seconde



Latence p95 (ms)



20 clients, médiane de 3 essais	Avant (N+1)	Après (groupé)	Gain
Débit (req/s)	246,9	1032,1	×4,2
p95 (ms)	98,0	26,8	−73 %
Erreurs	0	0	=
Les 3 essais (req/s)	238 à 253	978 à 1082	intervalles disjoints

À DIRE

Pour le savoir, je mesure avec 1, 5, 10 et 20 clients, trois essais alternés, un seul changement. Regardez la courbe de gauche : le débit de l'ancienne version reste plat autour de 250 ; celui de la nouvelle monte et se stabilise bien plus haut. À 20 clients, on passe de 247 à **1032 requêtes par seconde**, fois 4,2. À droite, le p95 passe de 98 à **26,8 millisecondes**. Zéro erreur. Le processeur ne bouge pas, autour de 50 pour cent : le gain vient du **travail évité**, pas d'une machine plus puissante. Et les trois essais de chaque version ne se chevauchent pas : c'est reproductible, pas un hasard de mesure.

Quel est le prix, et quelles limites ?

acte 5 : le prix du remède
à lire : environ 72 s · cumul ≈ 4 min 36 s

1 Symptôme

2 Enquête

3 Remède

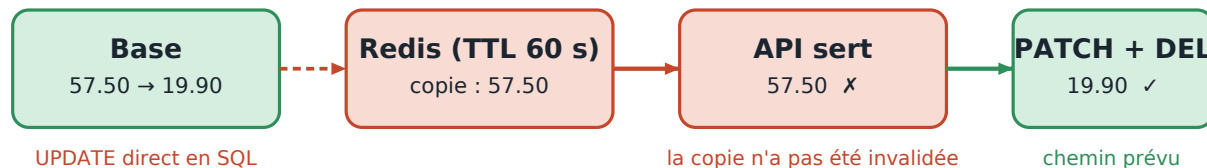
4 Charge

5 Prix

Fil rouge : Une page de 20 commandes paraît simple, mais devient lente quand plusieurs clients arrivent.
D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

Correction (même résultat, mêmes droits)	Résultat
JSON complet N+1 = groupé (5, 20, 50, 100 commandes)	identique
Sans jeton / client_id dans l'URL / curseur falsifié	401 / 400 / 400

FRAÎCHEUR : LE CACHE PEUT MENTIR



Cache : miss 2,7 ms (1 SELECT) → hit 0,99 ms (0 SQL) ; fraîcheur acceptée ≤ 60 s ; jamais de cache sur l'achat.

Variante rejetée	Pourquoi
Agrandir le pool 5 → 20	≈ 287 req/s contre 1032 (2 essais, indicatif)
PgBouncer pour accélérer	40 → 5 connexions mais 261 → 94 transactions/s

À DIRE

Premier prix, la fraîcheur. Si je mets un cache Redis devant, un hit ne fait aucune requête, mais la copie peut **mentir** : après une modification faite directement dans la base, l'API a servi **57.50 alors que la base disait 19.90**. Je borne la copie à 60 secondes, je l'invalides à chaque écriture par l'API, et je ne cache jamais l'achat.

Deuxième prix, ce que j'ai écarté : agrandir le pool n'atteint que 287 requêtes par seconde, sur deux essais, donc indicatif ; PgBouncer limite les connexions de 40 à 5 mais fait tomber le débit de 261 à 94. Il limite, il n'accélère pas. Risque restant : une lecture lente peut remettre une copie ancienne après l'invalidation. Réponse à ma question : le coût venait des allers-retours ; en les supprimant, le débit est multiplié par 4,2 sans erreur, mesuré sur cette machine, ce jeu et 1 à 20 clients, et pas encore en production.

Si on me pose la question

reproductibilité · conséquence pour l'API
à lire : environ 0 s · cumul ≈ 4 min 36 s

Question probable	Réponse courte
Comment reproduire sur un autre poste ?	même base (01_schema.sql, 02_donnees.sql, md5 comparés), PostgreSQL 18.6, Node 22, pool 5 ; charge fermée 1/5/10/20 clients, 3 essais alternés N+1 / groupé ; python3 atelier9/p2_charge.py (≈ 16 min) ; on compare l'ordre de grandeur
Que change la décision pour l'API ?	contrat inchangé (même JSON, mêmes droits) ; 21 → 2 requêtes donc moins de connexions occupées ; tableau d'ids borné à 100 ; avec le cache : copie ≤ 60 s et repli sur PostgreSQL si Redis tombe
Et si le gain disparaît ?	je le dirais : les séries pool et charge ouverte étaient instables, je les ai présentées essai par essai
Écart de protocole ?	ma mesure : 3 s d'échauffement et 10 s par essai ; la fiche cite 10 s et 30 s en exemple

À DIRE

Reproductibilité : même base, mêmes versions, même protocole ; je compare l'ordre de grandeur.

Conséquence pour l'API : contrat inchangé, moins de connexions occupées, fraîcheur bornée si cache, repli sur PostgreSQL si Redis tombe.

Phrase finale : « le coût venait des allers-retours ; mesuré ici, pas encore en production. »